

Chapter 6 Queues

The Queue Abstract Data Type (ADT)

◆ Prompt 1: What is a Queue ADT?

✓ Answer:

A **queue** is an abstract data type (ADT) that stores elements in a **sequence** with **restricted access**:

- **Insertion** → only at the **back**
- **Removal** → only from the **front**

✦ This enforces the **First-In, First-Out (FIFO)** rule.

◆ Prompt 2: What does FIFO mean?

✓ Answer:

FIFO (First-In, First-Out) means:

The first element inserted into the queue is the first one removed.

▀ Real-life examples:

- People waiting in line
- Print job queue
- Customer service requests

◆ Prompt 3: What are the two fundamental queue operations?

✓ Answer:

Operation	Description
enqueue (e)	Add element e to the back of the queue
dequeue ()	Remove and return the front element

⚠ Error occurs if `dequeue ()` is called on an empty queue.

Visual model of a queue

Diagram:

Front → [5 | 3 | 7 | 9] ← Back

↑ dequeue enqueue ↑

◆ Prompt 4: What supporting (accessor) methods does a queue provide?

✓ Answer:

Method	Purpose
<code>first()</code>	View front element (no removal)
<code>is_empty()</code>	Check if queue has no elements
<code>len(Q)</code>	Number of elements in queue

◆ Prompt 5: What assumptions are made about queues?

✓ Answer:

- A new queue starts **empty**
- No fixed capacity limit
- Elements can be of **any type**

◆ Prompt 6: How is a queue different from a stack?

✓ Answer:

Stack	Queue
LIFO	FIFO
Insert/remove at top	Insert at back, remove at front
push, pop	enqueue, dequeue

◆ Prompt 7: Step-by-step explanation of Example 6.4

Initial state:

Queue $Q = []$

▶ **Operation:** $Q.enqueue(5)$

[5]

▶ **Operation:** $Q.enqueue(3)$

[5, 3]

▶ **Operation:** $len(Q)$

- Return: 2

[5, 3]

▶ **Operation:** $Q.dequeue()$

- Removes 5
- Return: 5

[3]

▶ **Operation:** $Q.is_empty()$

- Return: False

[3]

▶ **Operation:** $Q.dequeue()$

- Removes 3
- Return: 3

[]

▶ **Operation:** $Q.is_empty()$

- Return: True

▶ **Operation:** $Q.dequeue()$

- ✗ Error (queue is empty)

▶ **Operation:** `Q.enqueue(7)`

[7]

▶ **Operation:** `Q.enqueue(9)`

[7, 9]

▶ **Operation:** `Q.first()`

- Return: 7

[7, 9]

▶ **Operation:** `Q.enqueue(4)`

[7, 9, 4]

▶ **Operation:** `len(Q)`

- Return: 3

▶ **Operation:** `Q.dequeue()`

- Removes 7

[9, 4]

◆ Prompt 8: Summary table (cleaned)

Operation	Return	Queue Contents
enqueue(5)	–	[5]
enqueue(3)	–	[5, 3]
len(Q)	2	[5, 3]
dequeue()	5	[3]
is_empty()	False	[3]
dequeue()	3	[]
is_empty()	True	[]
dequeue()	error	[]
enqueue(7)	–	[7]
enqueue(9)	–	[7, 9]
first()	7	[7, 9]
enqueue(4)	–	[7, 9, 4]
len(Q)	3	[7, 9, 4]
dequeue()	7	[9, 4]

◆ Prompt 9: Time complexity of queue operations

✓ Answer (ideal implementation):

Operation Time

enqueue $O(1)$

dequeue $O(1)$

first $O(1)$

is_empty $O(1)$

len $O(1)$

★ Achieved using:

- Circular array
- Linked list
- `collections.deque`

Array-Based Queue Implementation

◆ Prompt 1: Can we implement a queue using a Python list directly?

✓ Answer:

Yes, we **can** use a Python list:

- `enqueue(e) → list.append(e)`
- `dequeue() → list.pop(0)`

◆ Prompt 2: Why is `pop(0)` inefficient?

✓ Answer:

When you call `pop(0)`, Python **shifts all subsequent elements left by one** to fill the “hole” at index 0.

- Worst-case time complexity: $\Theta(n)$
- Example: If the list has 1,000 elements, 999 elements must be shifted every dequeue.

◆ Prompt 3: How can we improve dequeuing efficiency?

✓ Answer:

Instead of removing the first element physically, we:

1. Maintain a **front index variable** f to track the front of the queue.
2. Replace dequeued elements with `None`.
3. Increment f on each dequeue.

This avoids shifting elements and makes **dequeue** $O(1)$.

◆ Prompt 4: Visualize front index drift

📌 Example

Initial queue: [E, F, G, K, L, M], $f = 0$

Index: 0 1 2 3 4 5

Queue: [E, F, G, K, L, M]

Front: $f \rightarrow 0$

After one dequeue (`dequeue()` removes E):

Index: 0 1 2 3 4 5

Queue: [None, F, G, K, L, M]

Front: $f \rightarrow 1$

After two more dequeues (F and G):

Index: 0 1 2 3 4 5

Queue: [None, None, None, K, L, M]

Front: $f \rightarrow 3$

✓ Front “drifts” away from 0 without shifting elements.

◆ Prompt 5: What is the drawback of this approach?

✓ Answer:

- **Memory growth problem:**
 - The list length continues to grow with every enqueue, even if the current number of elements is small.
 - Example: A queue may hold 30 elements at any time, but after thousands of enqueues/dequeues, the underlying list could have length 10,000.
- **Consequence:**
 - Wasted memory
 - Poor cache performance
 - Not ideal for long-running applications

Python Queue Implementation

This is a **complete implementation of a queue ADT** using a Python list as a circular buffer.

◆ Prompt 1: What instance variables does `ArrayQueue` maintain?

✓ Answer:

1. **data** → The underlying Python list that stores elements.
2. **size** → The current number of elements in the queue (not the length of `data`).
3. **front** → Index of the first element in the queue.
 - Initially, `size = 0` and `front = 0`.
 - The list `data` is pre-allocated with a moderate capacity (`DEFAULT_CAPACITY = 10`).

◆ Prompt 2: How do we check if the queue is empty?

✓ Answer:

```
def isempty(self):  
    return self.size == 0
```

- Returns `True` if no elements exist.
- Runs in **O(1)**.

◆ Prompt 3: How do we access the front element?

✓ Answer:

```
def first(self):  
    if self.isempty():  
        raise Empty("Queue is empty")  
    return self.data[self.front]
```

- Uses `self.front` to get the first element **without removing it**.
- Raises an `Empty` exception if the queue is empty.

◆ Prompt 4: How does `dequeue()` work?

✓ Answer:

```
def dequeue(self):  
    if self.isempty():  
        raise Empty("Queue is empty")  
  
    answer = self.data[self.front]      # store the front element  
  
    self.data[self.front] = None        # remove reference for garbage  
collection  
  
    self.front = (self.front + 1) % len(self.data) # circular increment  
  
    self.size -= 1  
  
    return answer
```

- **Step 1:** Retrieve element at `self.front`.
- **Step 2:** Set slot to `None`.
- **Step 3:** Move `front` forward **circularly** using modulo.
- **Step 4:** Decrement `size`.

Time Complexity: $O(1)$

◆ Prompt 5: How does `enqueue(e)` work?

✓ Answer:

```
def enqueue(self, e):  
    if self.size == len(self.data):  
        self.resize(2 * len(self.data))    # double array size if full  
  
    avail = (self.front + self.size) % len(self.data)    # calculate next  
    available index  
  
    self.data[avail] = e  
  
    self.size += 1
```

- Computes next free index as `(front + size) % len(data)`.
- Inserts element there.
- If array is full, calls `resize()` first.
- **Time Complexity:** $O(1)$ amortized (due to occasional resizing).

◆ Prompt 6: How does resizing work?

✓ Answer:

```
def resize(self, cap):  
    old = self.data  
  
    self.data = [None] * cap  
  
    walk = self.front  
  
    for k in range(self.size):  
        self.data[k] = old[walk]    # copy elements to new array  
  
        walk = (walk + 1) % len(old)    # circular wrap-around  
  
    self.front = 0    # reset front to 0
```

- Creates a new list of size `cap`.
- Copies current elements **in order**.
- Resets `front` to 0.

Time Complexity: $O(n)$, but happens rarely $\rightarrow$ amortized $O(1)$ for `enqueue`.

◆ Prompt 7: Visualizing the circular queue

Example Operations:

1. `enqueue(10), enqueue(20), enqueue(30)`
2. `data = [10, 20, 30, None, None, None, None, None, None, None]`
3. `front = 0, size = 3`
- 4.
5. `dequeue()` → **removes 10**
6. `data = [None, 20, 30, None, None, None, None, None, None, None]`
7. `front = 1, size = 2`
- 8.
9. `enqueue(40)`
10. `data = [None, 20, 30, 40, None, None, None, None, None, None]`
11. `front = 1, size = 3`
- 12.
13. `dequeue()` → **removes 20**
14. `data = [None, None, 30, 40, None, None, None, None, None, None]`
15. `front = 2, size = 2`
- 16.

✓ Notice **circular movement** of `front` index.